

Lecture 4: Value Function Approximation

Bolei Zhou

<https://github.com/zhoubolei/introRL>

April 4, 2020

Today's Plan

- 1 Introduction on function approximation
- 2 Value function approximation for prediction
- 3 Value function approximation for control
- 4 Deep Q networks

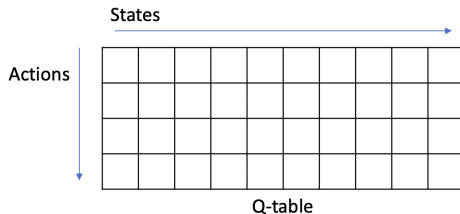
Introduction: Scaling up RL

- ➊ Previous lectures on small RL problems:
 - ➊ Cliff walk: 4×16 states
 - ➋ Mountain car: 1600 states
 - ➌ Tic-Tac-Toe: 10^3 states
- ➋ Large-scale problems:
 - ➊ Backgammon: 10^{20} states
 - ➋ Chess: 10^{47} states
 - ➌ Game of Go: 10^{170} states
 - ➍ Robot Arm and Helicopter have continuous state space
 - ➎ Number of atoms in universe: 10^{80}
- ➌ Challenge: How can we scale up the model-free methods for prediction and control?

Introduction: Scaling up RL

① In tabular methods we represent value function by a lookup table:

- ① Every state s has an entry $V(s)$
- ② Every state-action pair s, a has an entry $Q(s, a)$



② Challenges with large MDPs:

- ① too many states or actions to store in memory
- ② too slow to learn the value of each state individually

Scaling up RL with Function Approximation

- ① How to avoid explicitly learning or storing for every single state:
 - ① Dynamics or reward model
 - ② Value function, state-action function
 - ③ Policy
- ② Solution: Estimate with function approximation

$$\hat{v}(s, \mathbf{w}) \approx v^{\pi}(s)$$

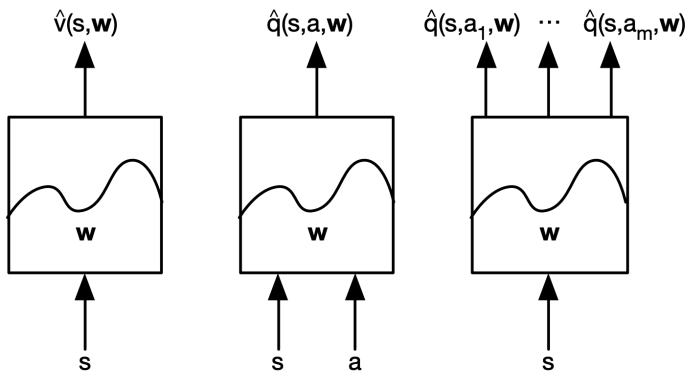
$$\hat{q}(s, a, \mathbf{w}) \approx q^{\pi}(s, a)$$

$$\hat{\pi}(a, s, \mathbf{w}) \approx \pi(a|s)$$

- ① Generalize from seen states to unseen states
- ② Update the parameter $\mathbf{w}$ using MC or TD learning

Types of value function approximation

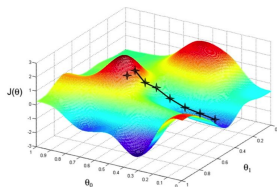
Several function designs:



Function Approximators

- ① Many possible function approximators:
 - ① Linear combinations of features
 - ② Neural networks
 - ③ Decision trees
 - ④ Nearest neighbors
- ② We will focus on function approximators that are differentiable
 - ① Linear feature representations
 - ② Neural networks

Review on Gradient Descend



- 1 Consider a function $J(\mathbf{w})$ that is a differentiable function of a parameter vector $\mathbf{w}$
- 2 Goal is to find parameter $\mathbf{w}^*$ that minimizes J
- 3 Define the gradient of $J(\mathbf{w})$ to be

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \left(\frac{\partial J(\mathbf{w})}{\partial w_1}, \frac{\partial J(\mathbf{w})}{\partial w_2}, \dots, \frac{\partial J(\mathbf{w})}{\partial w_n} \right)^T$$

- 4 Adjust $\mathbf{w}$ in the direction of the negative gradient, where α is step-size

$$\Delta \mathbf{w} = -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w})$$

Value Function Approximation with an Oracle

- 1 We assume that we have the oracle for knowing the true value for $v^\pi(s)$ for any given state s
- 2 Then the objective is to find the best approximate representation of $v^\pi(s)$
- 3 Thus use the mean squared error and define the loss function as

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[(v^\pi(s) - \hat{v}(s, \mathbf{w}))^2 \right]$$

- 4 Follow the gradient descend to find a local minimum

$$\begin{aligned}\Delta \mathbf{w} &= -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w}) \\ \mathbf{w}_{t+1} &= \mathbf{w}_t + \Delta \mathbf{w}\end{aligned}$$

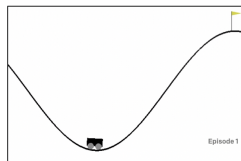
Representing State with Feature Vectors

1 Represent state using a feature vector

$$\mathbf{x}(s) = (x_1(s), \dots, x_n(s))^T$$

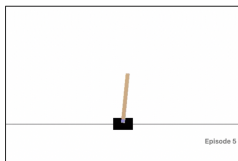
2 For example:

Mountain Car



Position of car, velocity of car

Cart Pole



Position of cart, velocity of cart,
angle of pole, rotation rate of pole

Game of Go in AlphaGo

Extended Data Table 2 | Input features for neural networks

Feature	# of planes	Description
Stone colour	3	Player stone / opponent stone / empty
Ones	1	A constant plane filled with 1
Turns since	8	How many turns since a move was played
Liberties	8	Number of liberties (empty adjacent points)
Capture size	8	How many opponent stones would be captured
Self-atari size	8	How many of own stones would be captured
Liberties after move	8	Number of liberties after this move is played
Ladder capture	1	Whether a move at this point is a successful ladder capture
Ladder escape	1	Whether a move at this point is a successful ladder escape
Sensibleness	1	Whether a move is legal and does not fill its own eyes
Zeros	1	A constant plane filled with 0
Player color	1	Whether current player is black

48 places of 19x19 feature maps

Linear Value Function Approximation

- 1 Represent value function by a linear combination of features

$$\hat{v}(s, \mathbf{w}) = \mathbf{x}(s)^T \mathbf{w} = \sum_{j=1}^n x_j(s) w_j$$

- 2 The objective function is quadratic in parameter $\mathbf{w}$

$$J(\mathbf{w}) = \mathbb{E}_{\pi} \left[(v^{\pi}(s) - \mathbf{x}(s)^T \mathbf{w})^2 \right]$$

- 3 Thus the update rule is as simple as

$$\Delta \mathbf{w} = \alpha (v^{\pi}(s) - \hat{v}(s, \mathbf{w})) \mathbf{x}(s)$$

Update = StepSize $\times$ PredictionError $\times$ FeatureValue

- 4 Stochastic gradient descent converges to global optimum. Because in the linear case, there is only one optimum, thus local optimum is automatically converge to or near the global optimum.

Linear Value Function Approximation with Table Lookup Feature

- 1 Table lookup is a special case of linear value function approximation
- 2 Table lookup feature is one-hot vector as follows

$$\mathbf{x}^{table}(s) = (\mathbf{1}(s = s_1), \dots, \mathbf{1}(s = s_n))^T$$

- 3 Then we can see that each element on the parameter vector $\mathbf{w}$ indicates the value of each individual state

$$\hat{v}(s, \mathbf{w}) = (\mathbf{1}(s = s_1), \dots, \mathbf{1}(s = s_n)) (w_1, \dots, w_n)^T$$

- 4 Thus we have $\hat{v}(s_k, \mathbf{w}) = w_k$

Value Function Approximation for Model-free Prediction

- ① In practice, no access to oracle of the true value $v^\pi(s)$ for any state s
- ② Recall model-free prediction
 - ① Goal is to evaluate v^π following a fixed policy π
 - ② A lookup table is maintained to store estimates v^π or q^π
 - ③ Estimates are updated after each episode (MC method) or after each step (TD method)
- ③ Thus what we can do is to **include the function approximation step in the loop**

Incremental VFA Prediction Algorithms

- ① We assumed that true value function $v^\pi(s)$ given by supervisor/oracle

$$\Delta \mathbf{w} = \alpha \left(v^\pi(s) - \hat{v}(s, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{v}(s_t, \mathbf{w})$$

- ② But in RL there is no supervisor, only rewards
- ③ In practice, we substitute the **target** for $v^\pi(s)$

- ① For MC, the target is the actual return G_t

$$\Delta \mathbf{w} = \alpha \left(G_t - \hat{v}(s_t, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{v}(s_t, \mathbf{w})$$

- ② For TD(0), the target is the TD target $R_{t+1} + \gamma \hat{v}(s_{t+1}, \mathbf{w})$

$$\Delta \mathbf{w} = \alpha \left(R_{t+1} + \gamma \hat{v}(s_{t+1}, \mathbf{w}) - \hat{v}(s_t, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{v}(s_t, \mathbf{w})$$

Monte-Carlo Prediction with VFA

- 1 Return G_t is an **unbiased**, but **noisy** sample of true value $v^\pi(s_t)$
- 2 Why unbiased? $\mathbb{E}[G_t] = v^\pi(s_t)$
- 3 So we have the training data that can be used for supervised learning in VFA:

$$\langle S_1, G_1 \rangle, \langle S_2, G_2 \rangle, \dots, \langle s_t, G_T \rangle$$

- 4 Using linear Monte-Carlo policy evaluation

$$\begin{aligned}\Delta \mathbf{w} &= \alpha \left(G_t - \hat{v}(s_t, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{v}(s_t, \mathbf{w}) \\ &= \alpha \left(G_t - \hat{v}(s_t, \mathbf{w}) \right) \mathbf{x}(s_t)\end{aligned}$$

- 5 Monte-Carlo prediction converges, in both linear and non-linear value function approximation.

TD Prediction with VFA

- ① TD target $R_{t+1} + \gamma \hat{v}(s_{t+1}, \mathbf{w})$ is a **biased** sample of true value $v^\pi(s_t)$
- ② Why biased? It is drawn from our previous estimate, rather than the true value: $\mathbb{E}[R_{t+1} + \gamma \hat{v}(s_{t+1}, \mathbf{w})] \neq v^\pi(s_t)$
- ③ We have the training data used for supervised learning in VFA:

$$\langle S_1, R_2 + \gamma \hat{v}(s_2, \mathbf{w}) \rangle, \langle S_2, R_3 + \gamma \hat{v}(s_3, \mathbf{w}) \rangle, \dots, \langle S_{T-1}, R_T \rangle$$

- ④ Using linear TD(0), the stochastic gradient descend update is

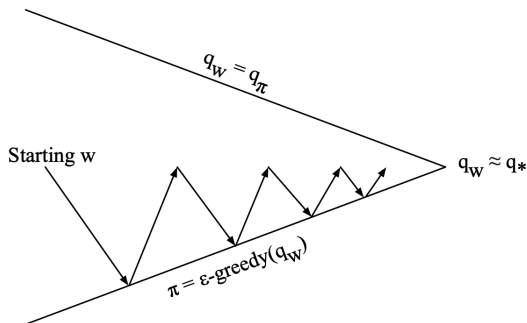
$$\begin{aligned}\Delta \mathbf{w} &= \alpha \left(R + \gamma \hat{v}(s', \mathbf{w}) - \hat{v}(s, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w}) \\ &= \alpha \left(R + \gamma \hat{v}(s', \mathbf{w}) - \hat{v}(s, \mathbf{w}) \right) \mathbf{x}(s)\end{aligned}$$

This is also called as semi-gradient, as we ignore the effect of changing the weight vector $\mathbf{w}$ on the target

- ⑤ Linear TD(0) converges(close) to global optimum

Control with Value Function Approximation

Generalized policy iteration



- 1 Policy evaluation: **approximate** policy evaluation, $\hat{q}(\cdot, \cdot, \mathbf{w}) \approx q^\pi$
- 2 Policy improvement: ϵ -greedy policy improvement

Action-Value Function Approximation

- 1 Approximate the action-value function

$$\hat{q}(s, a, \mathbf{w}) \approx q^\pi(s, a)$$

- 2 Minimize the MSE (mean-square error) between approximate action-value and true action-value (assume oracle)

$$J(\mathbf{w}) = \mathbb{E}_\pi[(q^\pi(s, a) - \hat{q}(s, a, \mathbf{w}))^2]$$

- 3 Stochastic gradient descend to find a local minimum

$$\Delta \mathbf{w} = \alpha (q^\pi(s, a) - \hat{q}(s, a, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(s, a, \mathbf{w})$$

Linear Action-Value Function Approximation

- 1 Represent state and action using a feature vector

$$\mathbf{x}(s, a) = (x_1(s, a), \dots, x_n(s, a))^T$$

- 2 Represent action-value function by a linear combination of features

$$\hat{q}(s, a, \mathbf{w}) = \mathbf{x}(s, a)^T \mathbf{w} = \sum_{j=1}^n x_j(s, a) w_j$$

- 3 Thus the stochastic gradient descend update

$$\Delta \mathbf{w} = \alpha (q^\pi(s, a) - \hat{q}(s, a, \mathbf{w})) \mathbf{x}(s, a)$$

Incremental Control Algorithm

Same to the prediction, there is no oracle for the true value $q^\pi(s, a)$, so we substitute a target

- 1 For MC, the target is return G_t

$$\Delta \mathbf{w} = \alpha (\textcolor{red}{G}_t - \hat{q}(s_t, a_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w})$$

- 2 For Sarsa, the target is TD target $R_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, \mathbf{w})$

$$\Delta \mathbf{w} = \alpha (\textcolor{red}{R}_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w})$$

- 3 For Q-learning, the target is TD target $R_{t+1} + \gamma \max_a \hat{q}(s_{t+1}, a, \mathbf{w})$

$$\Delta \mathbf{w} = \alpha (\textcolor{red}{R}_{t+1} + \gamma \max_a \hat{q}(s_{t+1}, a, \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w})$$

Semi-gradient Sarsa for VFA Control

Episodic Semi-gradient Sarsa for Estimating $\hat{q} \approx q_*$

Input: a differentiable function $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Repeat (for each episode):

$S, A \leftarrow$ initial state and action of episode (e.g., ε -greedy)

 Repeat (for each step of episode):

 Take action A , observe R, S'

 If S' is terminal:

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$

 Go to next episode

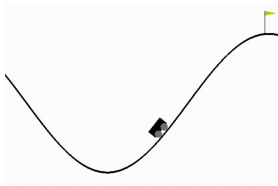
 Choose A' as a function of $\hat{q}(S', \cdot, \mathbf{w})$ (e.g., ε -greedy)

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$

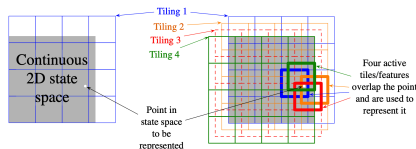
$S \leftarrow S'$

$A \leftarrow A'$

Mountain Car Example



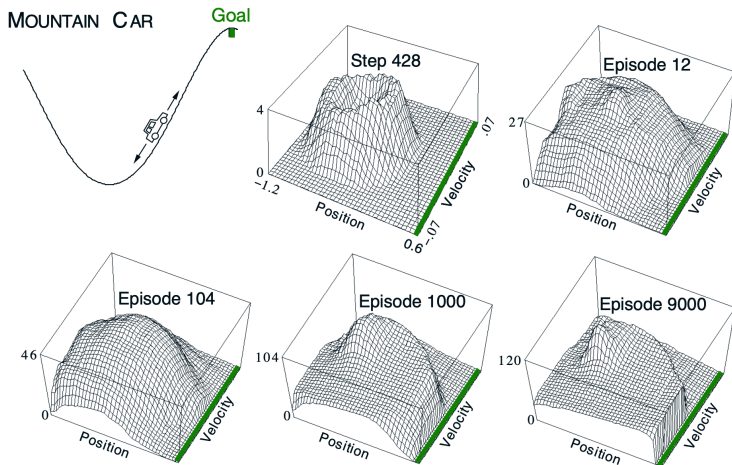
- ① Control task: move the car up the hill with forward and backward acceleration
 - ① Continuous state: [position of the car, velocity of the car]
 - ② Action: [full throttle forward, full throttle backward, zero throttle]
- ② Grid-tiling coding of the continuous 2D state space with 4 tilings



- ③ Q function approximator: $\hat{q}(s, a, \mathbf{w}) = \mathbf{w}^T \mathbf{x}(s, a) = \sum_{i=1}^4 w_i x_i(s, a)$

Mountain Car Example

- 1 Visualization of cost-to-go function ($-\max_a \hat{q}(s, a, \mathbf{w})$) learned over episodes



Mountain Car Example

- 1 Code example: https://github.com/cuhkrlcourse/RLexample/blob/master/modelfree/q_learning_mountaincar.py

Convergence of Control Methods with VFA

- 1 For Sarsa,

$$\Delta \mathbf{w} = \alpha (R_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w})$$

- 2 For Q-learning,

$$\Delta \mathbf{w} = \alpha (R_{t+1} + \gamma \max_a \hat{q}(s_{t+1}, a, \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w})$$

- 3 TD with VFA doesn't follow the gradient of any objective function
- 4 The updates involve doing an approximate Bellman backup followed by fitting the underlying value function
- 5 That is why TD can diverge when off-policy or using non-linear function approximation
- 6 Challenge for off-policy control: behavior policy and target policy are not identical, thus value function approximation can diverge

The Deadly Triad for the Danger of Instability and Divergence

Potential problematic issues:

- ➊ **Function approximation:** A scalable way of generalizing from a state space much larger than the memory and computational resources
- ➋ **Bootstrapping:** Update targets that include existing estimates (as in dynamic programming or TD methods) rather than relying exclusively on actual rewards and complete returns (as in MC methods)
- ➌ **Off-policy training:** training on a distribution of transitions other than that produced by the target policy
- ➍ See details at Textbook Chapter 11.3

Convergence of Control Methods

Algorithm	Table Lookup	Linear	Non-Linear
Monte-Carlo Control	✓	(✓)	X
Sarsa	✓	(✓)	X
Q-Learning	✓	X	X

(✓) moves around the near-optimal value function

Batch Reinforcement Learning

- ① Incremental gradient descend update is simple
- ② But it is not sample efficient
- ③ Batch-based methods seek to find the best fitting value function for a batch of the agent's experience

Least Square Prediction

- 1 Given the value function approximation $\hat{v}(s, \mathbf{w}) \approx v^\pi(s)$
- 2 The experience $\mathcal{D}$ consisting of $\langle \text{state}, \text{value} \rangle$ pairs (may from one episode or many previous episodes)

$$\mathcal{D} = \{ \langle s_1, v_1^\pi \rangle, \dots, \langle s_T, v_T^\pi \rangle \}$$

- 3 Objective: To optimize the parameter $\mathbf{w}$ that best fit all the experience $\mathcal{D}$
- 4 Least squares algorithms are used to minimize the sum-squared error between $\hat{v}(s_t, \mathbf{w})$ and the target values v_t^π

$$\begin{aligned} \mathbf{w}^* &= \arg \min_{\mathbf{w}} \mathbb{E}_{\mathcal{D}} [(v^\pi - \hat{v}(s, \mathbf{w}))^2] \\ &= \arg \min_{\mathbf{w}} \sum_{t=1}^T (v_t^\pi - \hat{v}(s_t, \mathbf{w}))^2 \end{aligned}$$

Stochastic Gradient Descent with Experience Replay

- 1 Given the experience consisting of $\langle state, value \rangle$ pairs

$$\mathcal{D} = \{ \langle s_1, v_1^\pi \rangle, \dots, \langle s_t, v_t^\pi \rangle \}$$

- 2 Iterative solution could be to repeat the following two steps

- 1 Randomly sample one pair $\langle state, value \rangle$ from the experience

$$\langle s, v^\pi \rangle \sim \mathcal{D}$$

- 2 Apply the stochastic gradient descent update

$$\Delta \mathbf{w} = \alpha (v^\pi - \hat{v}(s, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w})$$

- 3 The solution from the gradient descend method converges to the least squares solution

$$\mathbf{w}^{LS} = \arg \min_{\mathbf{w}} \sum_{t=1}^T (v_t^\pi - \hat{v}(s_t, \mathbf{w}))^2$$

Summary

- ① Today's content:
 - ① Value function approximation for prediction
 - ② Value function approximation for control
 - ③ Least square prediction and control
- ② Next part: Deep Q-Learning
 - ① DeepMind's **Nature** paper: Mnih, Volodymyr; et al. (2015).
Human-level control through deep reinforcement learning
 - ② Entering the period of modern RL: deep learning + reinforcement learning

Scaling up RL

- ➊ Generalization of RL to tackle practical problems such as self-driving cars, Atari, consumer marketing, healthcare, education
- ➋ Most of these practical problems have enormous state and/or action spaces
- ➌ It requires the representations of models/values/policies that can generalize across states and/or actions
- ➍ Solution: to represent a value function with a parameterized function instead of a lookup table

$$\begin{aligned}\hat{v}(s, \mathbf{w}) &\approx v^{\pi}(s) \\ \hat{q}(s, a, \mathbf{w}) &\approx q^{\pi}(s, a)\end{aligned}$$

Linear Function Approximation

- 1 Represent value function by a linear combination of features

$$\hat{v}(s, \mathbf{w}) = \mathbf{x}(s)^T \mathbf{w} = \sum_{j=1}^n x_j(s) w_j$$

- 2 Objective function is $J(\mathbf{w}) = \mathbb{E}_{\pi} \left[(v^{\pi}(s) - \mathbf{x}(s)^T \mathbf{w})^2 \right]$
- 3 Update is as simple as $\Delta \mathbf{w} = \alpha (v^{\pi}(s) - \hat{v}(s, \mathbf{w})) \mathbf{x}(s)$
- 4 But there is no oracle for the true value $v^{\pi}(s)$, we substitute with the target from MC or TD
 - 1 for MC policy evaluation,

$$\Delta \mathbf{w} = \alpha \left(G_t - \hat{v}(s_t, \mathbf{w}) \right) \mathbf{x}(s_t)$$

- 2 for TD policy evaluation,

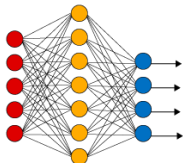
$$\Delta \mathbf{w} = \alpha \left(R_{t+1} + \gamma \hat{v}(s_{t+1}, \mathbf{w}) - \hat{v}(s_t, \mathbf{w}) \right) \mathbf{x}(s_t)$$

Linear vs Nonlinear Value Function Approximation

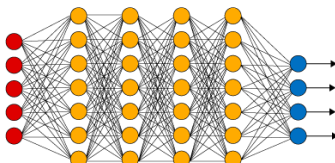
- ① Linear VFA often works well given the right set of features
- ② But it requires manual designing of the feature set
- ③ Alternative is to use a much richer function approximator that is able to directly learn from states without requiring the feature design
- ④ Nonlinear function approximator: Deep neural networks

Deep Neural Networks

Simple Neural Network



Deep Learning Neural Network



● Input Layer

● Hidden Layer

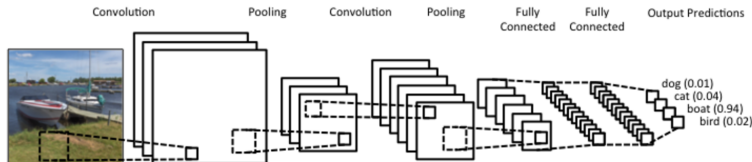
● Output Layer

- 1 Multiple layers of linear functions, with non-linear operators between layers

$$f(\mathbf{x}; \theta) = \mathbf{W}_{L+1}^T \sigma(\mathbf{W}_L^T \sigma(\dots \sigma(\mathbf{W}_1^T \mathbf{x} + \mathbf{b}_1) + \dots + \mathbf{b}_{L-1}) + \mathbf{b}_L) + \mathbf{b}_{L+1}$$

- 2 The chain rule to backpropagate the gradient to update the weights using the loss function $L(\theta) = \frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - f(\mathbf{x}; \theta) \right)^2$

Convolutional Neural Networks



- 1 Convolution encodes the local information in 2D feature map
- 2 Layers of convolution, reLU, batch normalization, etc.
- 3 CNNs are widely used in computer vision (more than 70% top conference papers using CNNs)
- 4 A detailed introduction on CNNs:
<http://cs231n.github.io/convolutional-networks/>

Deep Reinforcement Learning

- ① Frontier in machine learning and artificial intelligence
- ② Deep neural networks are used to represent
 - ① Value function
 - ② Policy function (policy gradient methods to be introduced)
 - ③ World model
- ③ Loss function is optimized by stochastic gradient descent (SGD)
- ④ Challenges
 - ① Efficiency: too many model parameters to optimize
 - ② The Deadly Triad for the danger of instability and divergence in training
 - ① Nonlinear function approximation
 - ② Bootstrapping
 - ③ Off-policy training

Deep Q-Networks (DQN)

- 1 DeepMind's **Nature** paper: Mnih, Volodymyr; et al. (2015).
Human-level control through deep reinforcement learning
- 2 DQN represents the action value function with neural network approximator
- 3 DQN reaches a professional human gaming level across many Atari games using the same network and hyperparameters



4 Atari Games: Breakout, Pong, Montezuma's Revenge, Private Eye

Recall: Action-Value Function Approximation

- 1 Approximate the action-value function

$$\hat{q}(s, a, \mathbf{w}) \approx q^\pi(s, a)$$

- 2 Minimize the MSE (mean-square error) between approximate action-value and true action-value (assume oracle)

$$J(\mathbf{w}) = \mathbb{E}_\pi[(q^\pi(s, a) - \hat{q}(s, a, \mathbf{w}))^2]$$

- 3 Stochastic gradient descend to find a local minimum

$$\Delta \mathbf{w} = \alpha(q^\pi(s, a) - \hat{q}(s, a, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(s, a, \mathbf{w})$$

Recall: Incremental Control Algorithm

Same to the prediction, there is no oracle for the true value $q^\pi(s, a)$, so we substitute a target

- ① For MC, the target is the return G_t

$$\Delta \mathbf{w} = \alpha (\textcolor{red}{G}_t - \hat{q}(s_t, a_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w})$$

- ② For Sarsa, the target is the TD target $R_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, \mathbf{w})$

$$\Delta \mathbf{w} = \alpha (\textcolor{red}{R}_{t+1} + \gamma \hat{q}(\textcolor{red}{s}_{t+1}, \textcolor{red}{a}_{t+1}, \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w})$$

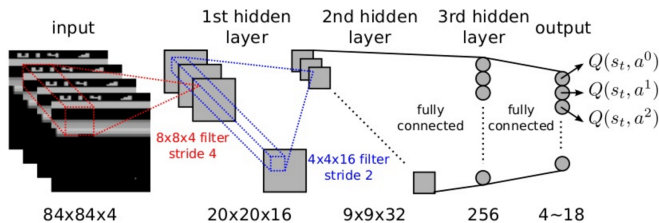
- ③ For Q-learning, the target is the TD target

$$R_{t+1} + \gamma \max_a \hat{q}(s_{t+1}, a, \mathbf{w})$$

$$\Delta \mathbf{w} = \alpha (\textcolor{red}{R}_{t+1} + \gamma \max_a \hat{q}(\textcolor{red}{s}_{t+1}, \textcolor{red}{a}, \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w})$$

DQN for Playing Atari Games

- 1 End-to-end learning of values $Q(s, a)$ from input pixel frame
- 2 Input state s is a stack of raw pixels from latest 4 frames
- 3 Output of $Q(s, a)$ is 18 joystick/button positions
- 4 Reward is the change in score for that step
- 5 Network architecture and hyperparameters fixed across all games

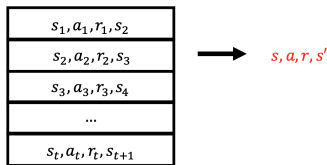


Q-Learning with Value Function Approximation

- ① Two of the issues causing problems:
 - ① Correlations between samples
 - ② Non-stationary targets
- ② Deep Q-learning (DQN) addresses both of these challenges by
 - ① Experience replay
 - ② Fixed Q targets

DQNs: Experience Replay

- 1 To reduce the correlations among samples, store transition (s_t, a_t, r_t, s_{t+1}) in replay memory $\mathcal{D}$



- 2 To perform experience replay, repeat the following
 - 1 sample an experience tuple from the dataset: $(s, a, r, s') \sim \mathcal{D}$
 - 2 compute the target value for the sampled tuple: $r + \gamma \max_{a'} \hat{Q}(s', a', \mathbf{w})$
 - 3 use stochastic gradient descent to update the network weights

$$\Delta \mathbf{w} = \alpha \left(r + \gamma \max_{a'} \hat{Q}(s', a', \mathbf{w}) - Q(s, a, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s, a, \mathbf{w})$$

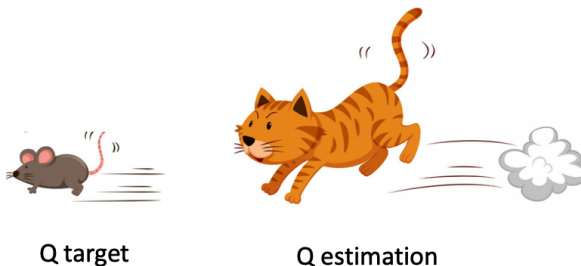
DQNs: Fixed Targets

- 1 To help improve stability, fix the target weights used in the target calculation for multiple updates
- 2 Let a different set of parameter $\mathbf{w}^-$ be the set of weights used in the target, and $\mathbf{w}$ be the weights that are being updated
- 3 To perform experience replay with fixed target, repeat the following
 - 1 sample an experience tuple from the dataset: $(s, a, r, s') \sim \mathcal{D}$
 - 2 compute the target value for the sampled tuple:
 $r + \gamma \max_{a'} \hat{Q}(s', a', \mathbf{w}^-)$
 - 3 use stochastic gradient descent to update the network weights

$$\Delta \mathbf{w} = \alpha \left(r + \gamma \max_{a'} \hat{Q}(s', a', \mathbf{w}^-) - Q(s, a, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s, a, \mathbf{w})$$

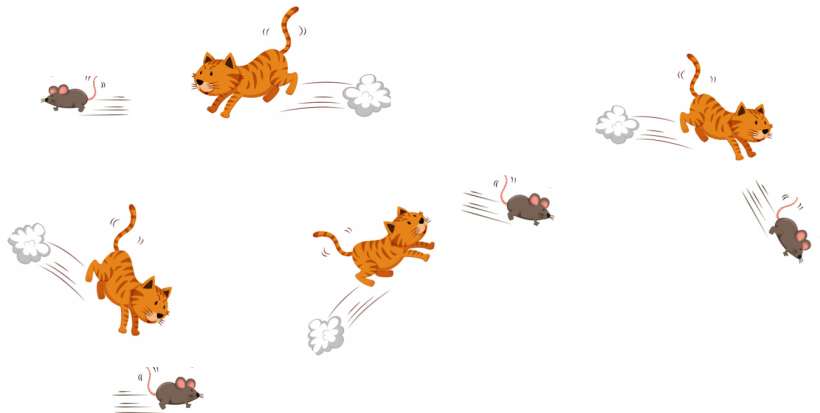
Why fixed target

- 1 In the original update, both Q estimation and Q target shifts at each time step
- 2 Imagine a cat (Q estimation) is chasing after a mouse (Q target)
- 3 The cat must reduce the distance to the mouse



Why fixed target

- 1 Both the cat and mouse are moving,



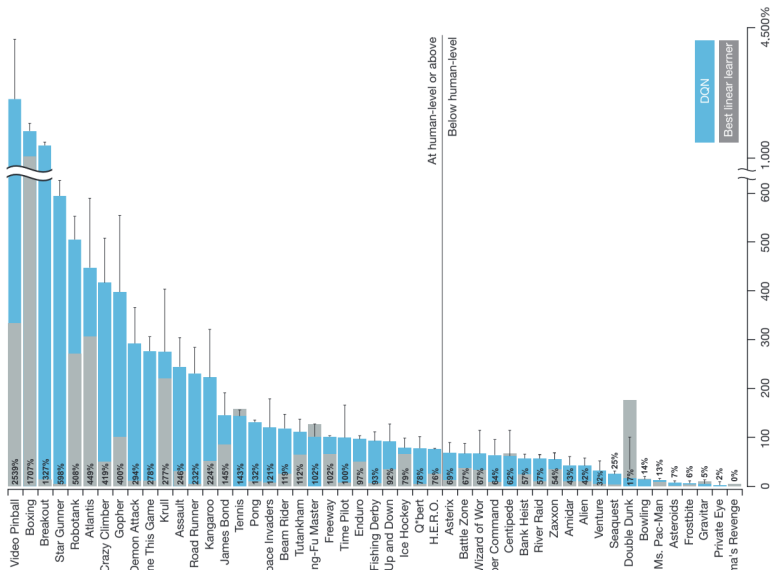
Why fixed target

- 1 This could lead to a strange path of chasing (an oscillated training history)



- 2 Solution: fix the target for a period of time during the training

Performance of DQNs on Atari



Abalation Study on DQNs

① Game score under difference conditions

	Replay Fixed-Q	Replay Q-learning	No replay Fixed-Q	No replay Q-learning
Breakout	316.81	240.73	10.16	3.17
Enduro	1006.3	831.25	141.89	29.1
River Raid	7446.62	4102.81	2867.66	1453.02
Seaquest	2894.4	822.55	1003	275.81
Space Invaders	1088.94	826.33	373.22	301.99

Demo of DQNs

- 1 Demo of Breakout by DQN:
<https://www.youtube.com/watch?v=V1eYniJ0Rnk>
- 2 Demo of Flappy Bird by DQN:
<https://www.youtube.com/watch?v=xM62SpKAZHU>
- 3 Code of DQN in PyTorch: <https://github.com/cuhkrlcourse/DeepRL-Tutorials/blob/master/01.DQN.ipynb>
- 4 Code of Flappy Bird:
<https://github.com/xmfbit/DQN-FlappyBird>

Summary of DQNs

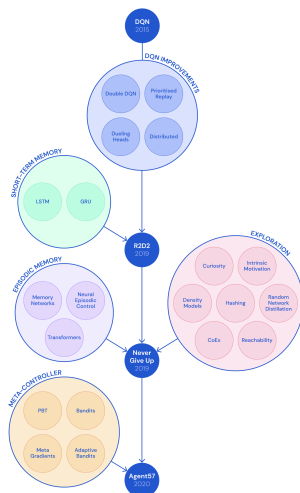
- 1 DQN uses experience replay and fixed Q-targets
- 2 Store transition $(s_t, a_t, r_{t+1}, s_{t+1})$ in replay memory $\mathcal{D}$
- 3 Sample random mini-batch of transitions (s, a, r, s') from $\mathcal{D}$
- 4 Compute Q-learning targets w.r.t. old, fixed parameters $\mathbf{w}^-$
- 5 Optimizes MSE between Q-network and Q-learning targets using stochastic gradient descent

Improving DQN

- ① Success in Atari has led to a huge excitement in using deep neural networks to do value function approximation in RL
- ② Many follow-up works on improving DQNs
 - ① **Double DQN**: Deep Reinforcement Learning with Double Q-Learning. Van Hasselt et al, AAAI 2016
 - ② **Dueling DQN**: Dueling Network Architectures for Deep Reinforcement Learning. Wang et al, best paper ICML 2016
 - ③ **Prioritized Replay**: Prioritized Experience Replay. Schaul et al, ICLR 2016
- ③ A nice tutorial on the relevant algorithms:
<https://github.com/cuhkrlcourse/DeepRL-Tutorials>

March 31, 2020: Agent57

- 1 <https://deepmind.com/blog/article/Agent57-Outperforming-the-human-Atari-benchmark>



Optional Homework

- 1 Go through the Jupyter tutorial and training your own gaming agent:
<https://github.com/cuhkrlcourse/DeepRL-Tutorials>
- 2 Optional Homework 2:
<https://github.com/cuhkrlcourse/ierg6130-assignment>
- 3 Next time: Policy-based RL

Additional slides for Least Squares Methods

- 1 Additional slides for Least Squares Methods

Solving Linear Least Squares Prediction

- ① Experience replay finds least squares solution, but it may take many iterations
- ② Using linear value function approximation $\hat{v}(s, \mathbf{w}) = \mathbf{x}(s)^T \mathbf{w}$, we can solve the least squares solution directly

Solving Linear Least Squares Prediction 2

- ① At minimum of $LS(\mathbf{w})$, the expected update must be zero

$$\mathbb{E}_{\mathcal{D}}[\Delta \mathbf{w}] = 0$$

- ② thus

$$\Delta \mathbf{w} = \alpha \sum_{t=1}^T \mathbf{x}(s_t)(v_t^{\pi} - \mathbf{x}(s_t)^T \mathbf{w}) = 0$$

$$\sum_{t=1}^T \mathbf{x}(s_t)v_t^{\pi} = \sum_{t=1}^T \mathbf{x}(s_t)\mathbf{x}(s_t)^T \mathbf{w}$$

$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(s_t)\mathbf{x}(s_t)^T \right)^{-1} \sum_{t=1}^T \mathbf{x}(s_t)v_t^{\pi}$$

- ③ For N features, the matrix inversion complexity is $O(N^3)$

Linear Least Squares Prediction Algorithms

- ① Again, we do not know the true values v_t^π
- ② In practice, the training data must use noisy or biased samples of v_t^π
 - ① LSMC: Least squares Monte-Carlo uses return
 $v_t^\pi \approx G_t$
 - ② LSTD: Least squares TD uses TD target
 $v_t^\pi \approx R_{t+1} + \gamma \hat{v}(s_{t+1}, \mathbf{w})$

Linear Least Squares Prediction Algorithms

$$\text{LSMC} \quad 0 = \sum_{t=1}^T \alpha (G_t - \hat{v}(s_t, \mathbf{w})) \mathbf{x}(s_t)$$

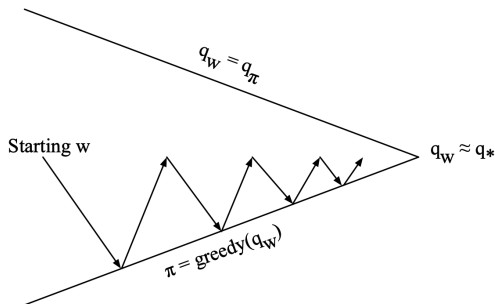
$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(s_t) \mathbf{x}(s_t)^T \right)^{-1} \sum_{t=1}^T \mathbf{x}(s_t) G_t$$

$$\text{LSTD} \quad 0 = \sum_{t=1}^T \alpha (R_{t+1} + \gamma \hat{v}(s_{t+1}, \mathbf{w}) - \hat{v}(s_t, \mathbf{w})) \mathbf{x}(s_t)$$

$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(s_t) (\mathbf{x}(s_t) - \gamma \mathbf{x}(s_{t+1}))^T \right)^{-1} \sum_{t=1}^T \mathbf{x}(s_t) R_{t+1}$$

Least Square Control with Value Function Approximation

Generalized policy iteration



- 1 Policy evaluation: Policy evaluation by **least squares Q-learning**
- 2 Policy improvement: greedy policy improvement

Least Squares Action-Value Function Approximation

- 1 Approximate action-value function $q^\pi(s, a)$
- 2 using linear combination of features $\mathbf{x}(s, a)$

$$\hat{q}(s, a, \mathbf{w}) = \mathbf{x}(s, a)^T \mathbf{w} \approx q^\pi(s, a)$$

- 3 minimize least squares error between $\hat{q}(s, a, \mathbf{w})$ and $q^\pi(s, a)$
- 4 The experience is generate from using policy π , consisting of $\langle \textit{state}, \textit{action}, \textit{action} - \textit{value} \rangle$

Least Squares Control

- 1 For policy evaluation we want to efficiently use all the experience
- 2 For control we want to improve the policy
- 3 The experience is generated from many policies (previous old policies)
- 4 So to evaluate $q^\pi(s, a)$ we must learn **off-policy**
- 5 We follow the same idea of Q-Learning:
 - 1 Use the experience generated by old policy:

$$s_t, a_t, R_{t+1}, S_{t+1} \sim \pi_{old}$$

- 2 Consider the alternative successor action $A' = \pi_{new}(s_{t+1})$ using greedy
- 3 Update $\hat{q}(s_t, a_t, \mathbf{w})$ towards value of alternative action $R_{t+1} + \gamma \hat{q}(s_{t+1}, A', \mathbf{w})$

Least Squares Q-Learning

- 1 Consider the following linear Q-learning update

$$\begin{aligned}\delta &= R_{t+1} + \gamma \hat{q}(s_{t+1}, \pi(s_{t+1}), \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w}) \\ \Delta \mathbf{w} &= \alpha \delta \mathbf{x}(s_t, a_t)\end{aligned}$$

- 2 LSTDQ algorithm: solve the total sum of the gradient = zero:

$$\begin{aligned}0 &= \sum_{t=1}^T \alpha (R_{t+1} + \gamma \hat{q}(s_{t+1}, \pi(s_{t+1}), \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w})) \mathbf{x}(s_t, a_t) \\ \mathbf{w} &= \left(\sum_{t=1}^T \mathbf{x}(s_t, a_t) (\mathbf{x}(s_t, a_t) - \gamma \mathbf{x}(s_{t+1}, \pi(s_{t+1})))^T \right)^{-1} \sum_{t=1}^T \mathbf{x}(s_t, a_t) R_{t+1}\end{aligned}$$

Least Squares Policy Iteration Algorithm

- 1 Use LSTDQ for policy evaluation
- 2 Repeatedly re-evaluate experience $\mathcal{D}$ with different policies

```
function LSPI-TD( $\mathcal{D}, \pi_0$ )  
   $\pi' \leftarrow \pi_0$   
  repeat  
     $\pi \leftarrow \pi'$   
     $Q \leftarrow \text{LSTDQ}(\pi, \mathcal{D})$   
    for all  $s \in \mathcal{S}$  do  
       $\pi'(s) \leftarrow \underset{a \in \mathcal{A}}{\operatorname{argmax}} Q(s, a)$   
    end for  
  until ( $\pi \approx \pi'$ )  
  return  $\pi$   
end function
```


Convergence of Control Methods

Algorithm	Table Lookup	Linear	Non-Linear
Monte-Carlo Control	✓	(✓)	X
Sarsa	✓	(✓)	X
Q-Learning	✓	X	X
LSPI	✓	(✓)	-

(✓) moves around the near-optimal value function

Additional slides for extending DQN

- 1 Additional slides for extending DQN

Improving DQN: Double DQN

- ① Handles the problem of the overestimation of Q-values
- ② Idea: use the two networks to decouple the action selection from the target Q value generation
- ③ Vanilla DQN:

$$\Delta \mathbf{w} = \alpha \left(r + \gamma \max_{a'} \hat{Q}(s', a', \mathbf{w}^-) - Q(s, a, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s, a, \mathbf{w})$$

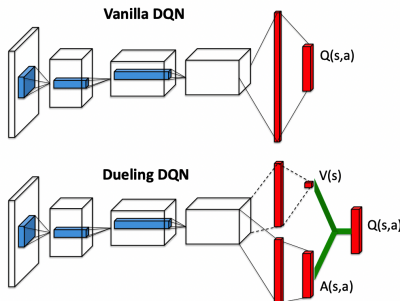
- ④ Double DQN:

$$\Delta \mathbf{w} = \alpha \left(r + \gamma \hat{Q}(s', \arg \max_{a'} Q(s', a', \mathbf{w}), \mathbf{w}^-) - Q(s, a, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s, a, \mathbf{w})$$

- ⑤ Code: https://github.com/cuhkrlcourse/DeepRL-Tutorials/blob/master/03.Double_DQN.ipynb

Improving DQN: Dueling DQN

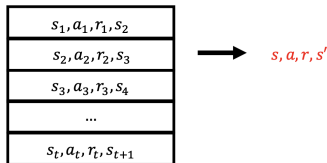
- 1 One branch estimates $V(s)$, other branch estimates the advantage for each action $A(s, a)$. Then $Q(s, a) = A(s, a) + V(s)$



- 2 By decoupling the estimation, intuitively the DuelingDQN can learn which states are (or are not) valuable without having to learn the effect of each action at each state
- 3 Code: https://github.com/cuhkrlcourse/DeepRL-Tutorials/blob/master/04.Dueling_DQN.ipynb

Improving DQN: Prioritized Experience Replay

- 1 Transition $(s_t, a_t, r_{t+1}, s_{t+1})$ is stored in and sampled from the replay memory $\mathcal{D}$



- 2 Priority is on the experience where there is a big difference between our prediction and the TD target, since it means that we have a lot to learn about it.
- 3 Define a priority score for each tuple i

$$p_i = |r + \gamma \max_{a'} Q(s_{i+1}, a', \mathbf{w}^-) - Q(s_i, a_i, \mathbf{w})|$$

- 4 Code: https://github.com/cuhkrlcourse/DeepRL-Tutorials/blob/master/06.DQN_PriorityReplay.ipynb

Improving over the DQN

- 1 Rainbow: Combining Improvements in Deep Reinforcement Learning. Matteo Hessel et al. AAAI 2018.
<https://arxiv.org/pdf/1710.02298.pdf>
- 2 It examines six extensions to the DQN algorithm and empirically studies their combination

